SC4050 Lab 5: A Parallel Stencil Solver

Lab Objectives

By the end of this lab, you will be able to:

- Build a 2D Cartesian process topology with `MPI_Dims_create`/`MPI_Cart_create` and locate a rank's four neighbors with `MPI_Cart_shift`.
- Construct and use a derived `MPI_Type_vector` datatype to exchange non-contiguous halo data
- Overlap halo-exchange communication with local computation using non-blocking `MPI_Isend`, `MPI_Irecv`, and `MPI_Waitall`
- Compare a blocking and an overlapped implementation's performance and explain the difference

Overview and Setup

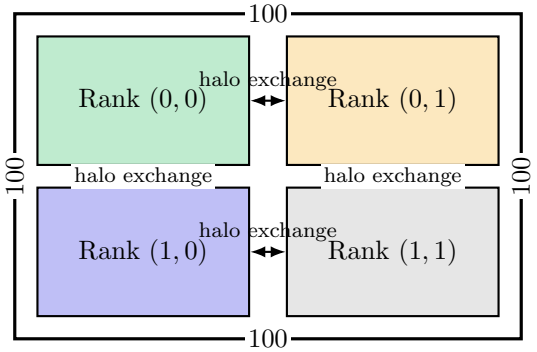
This is the capstone lab for the MPI block, combining one tool from each of Weeks 8-10 into a single working parallel program: a 2D heat-diffusion solver. You could use either your Raspberry cluster (recommended) or Aspire2A.

Two graded parts:

- **Lab experiments (5 marks)**: four tasks below, building one working program incrementally.
- **Lab quiz (5 marks)**: a 30-minute closed-book paper quiz at the end

The problem: a $G \times G$ grid of temperature values. All four physical edges of the *global* grid are held fixed at 100; every interior point starts at 0. Each iteration, every interior point is replaced by the average of its four neighbors (a Jacobi update). Run for long enough, and the whole grid should converge to a uniform 100, since a constant boundary has a constant steady state. You will decompose this grid across ranks using a 2D block decomposition on a Cartesian topology.

Global $G \times G$ grid



All four outer edges of the global grid are fixed at 100*; every interior cell starts at* 0*. Each rank owns one contiguous sub-block and exchanges one layer of halo (ghost) cells with each of its neighbors every iteration, so it can compute its own 5-point stencil update without ever touching another rank's memory directly.*

You are given `stencil_skeleton.c`, with five numbered `TODO` blocks corresponding to Tasks 1-3 below, and a working `jacobi()` helper function already written for you (the 5-point averaging formula).

Task 1: 2D Cartesian Decomposition Setup (0.5 marks)

Fill in `TODO 1`: build a 2D Cartesian topology with `MPI_Dims_create` and `MPI_Cart_create` (non-periodic), read back this rank's coordinates with `MPI_Cart_coords`, and find its four neighbors with two calls to `MPI_Cart_shift`.

Checkpoint: your program prints, for each rank, its coordinates and its up/down/left/right neighbor ranks, and these match what you'd expect by hand for at least two different values of `-np` (e.g. 4 and 8).

Task 2: Halo Exchange with Derived Datatypes (1.5 marks)

Fill in `TODO 2` (build and commit a column datatype with `MPI_Type_vector`) and `TODO 3` (post all non-blocking sends/receives for the top/bottom rows and left/right columns).

Checkpoint: your program (with Task 3's loop also filled in enough to actually call `MPI_Waitall` and iterate) runs stably for at least 100 iterations at `-np 4` without crashing or hanging.

Task 3: Iterative Solver with Overlap and a Convergence Check (1.5 marks)

Fill in `TODO 4` (the overlap: update the true interior *before* `MPI_Waitall`, then the outer ring *after* it) and `TODO 5` (compute this rank's largest change, then communicate across every rank to get the global largest change).

Run:

```
mpicc -o stencil stencil_skeleton.c -lm
mpirun -np 4 ./stencil 32 5000
```

Checkpoint:

- Every rank reports `converged=yes`, and the printed local average is close to 100.
- **Decomposition-independence check**: run the exact same program at `-np 1` and then at `-np 4` (or any other two divisor arrangements). Both runs must report the *same* iteration count and the same `global_max_diff`.

Task 4: Blocking vs. Overlapped Performance (1.5 marks)

Copy your working solution to a new file, `stencil_blocking.c`, and remove the overlap: call `MPI_Waitall` immediately after posting the 8 requests, *before* updating any cells at all, then update the entire local block (interior and outer ring together) in one pass.

Run both versions on a larger problem and with different number of MPI processes, e.g. `mpirun -np 8 ./stencil 256 200000` and the same for `./stencil_blocking`, under **two different placements**:

1. **Single node**: all ranks confined to one physical node.
2. **At least two nodes**: ranks spread across **at least 2** physical nodes, so some neighbor pairs actually communicate over the network, not through shared memory.

For *each* placement, report the total solve time for both versions and state whether you observe a measurable benefit from overlap. Then explain, using the Week 10 cost model, why the benefit does or does not appear in each case.

Checkpoint: both versions converge to the identical result (same iteration count, same `global_max_diff`) under both placements; you report the total solve time for both versions at both placements, state whether you see a benefit in each case, and your explanation correctly reasons about the cost-model difference between intra-node and inter-node communication.

Mark Summary

Task	**Marks**
Task 1: 2D Cartesian Decomposition Setup	0.5
Task 2: Halo Exchange with Derived Datatypes	1.5
Task 3: Iterative Solver with Overlap and Convergence	1.5
Task 4: Blocking vs. Overlapped Performance	1.5
Lab experiments total	**5.0**
Lab quiz (separate, 30 min, covers Weeks 8-10)	5.0
Lab total	**10.0**